Universidad Nacional Autónoma de México

Ingeniería en Computación

Compilers

Compiler

STUDENTS:
321184995
321331656
321058526
424138220
320560154

Group:
#05
Semester:
2026-II

Mexico, CDMX. 28th May 2026

Contents

1 Introduction

Compilers are fundamental tools in software development and computer programming because they allow the transformation of high-level source code into low-level instructions that can be directly executed by the processor. However, this process does not simply consist of translating text; it requires a sequence of stages responsible for analyzing, validating, and transforming the source code into an executable representation.

The compilation process is divided into several phases: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, and target code generation. Each stage fulfills a specific function and depends on the correct execution of the previous phase. In particular, lexical analysis represents the first phase of the compiler and is responsible for examining source code and dividing it into basic units called tokens. These tokens correspond to meaningful language elements such as reserved words, identifiers, constants, operators, and punctuation symbols.

However, identifying valid tokens is not sufficient to guarantee that a program is correctly constructed. Once the token sequence is obtained, it is necessary to verify that these tokens follow the grammatical structure defined for the programming language and that they also possess logical and semantic consistency. For this purpose, syntax analysis through a Parser and the application of Syntax Directed Translation (SDT) techniques are required to validate essential semantic rules such as type compatibility, valid variable declarations, and the proper construction of expressions and assignments.

The problem addressed in this project consists of the design and implementation of a compiler focused on processing .c source code files. The system must be capable of receiving programs written in the C programming language, performing lexical analysis to separate and classify lexemes, validating the syntactic structure through a Parser, and applying semantic rules using SDT. In this way, the compiler will be able to detect errors during the earliest phases of the compilation process and build a structured representation of the program.

Without tools such as the lexer, parser, and semantic analyzer, the compilation process would be incapable of understanding the intention behind the programmer's code, limiting itself to interpreting characters without structural meaning. Therefore, these stages constitute the fundamental basis that allows a sequence of text to be transformed into a logical and meaningful representation understandable by the system.

The system must follow a structured processing flow. First, the compiler receives a set of instructions contained in a C source code file. Subsequently, the input strings are processed through different analyzers, beginning with lexical analysis, followed by syntax and semantic analysis, in order to validate both the grammatical structure and the logical consistency of the program. Finally, once these stages are completed, the system generates a valid representation of the code that can be linked and prepared

for execution, displaying the corresponding results of the compilation process.

Understanding the internal mechanics of a compiler is a fundamental pillar in software systems development, as it reveals the exact processes through which a computer processes, interprets, and validates a programming language.

The primary motivation behind this project lies in breaking down and resolving the complexities of error detection prior to program execution. Lexical analysis, acting as the first line of defense, holds critical relevance: it not only streamlines the input stream by stripping away irrelevant elements like comments and whitespace, but it also establishes a clean structure of tokens that is indispensable for the success of subsequent phases.

Furthermore, constructing a parser and integrating a Syntax-Directed Translation (SDT) scheme allows high-abstraction concepts, such as context-free grammars, LR automata, ACTION/GOTO tables, and shift-reduce algorithms, to materialize into a fully functional software architecture. This hands-on implementation provides first-hand experience in how a system simultaneously validates both the grammatical structure and the logical coherence of a program.

General Objective:

Design and implement a compiler capable of processing source code files written in C language. The system will perform lexical, syntactic, and semantic analyses to validate the grammatical structure, logical coherence, and correct construction of the input program.

Specific Objectives:

- Develop a lexical analyzer to identify and classify source code components into specific tokens, such as keywords, identifiers, operators, constants, and punctuation marks, while filtering out irrelevant elements like whitespace and comments.
- Design an efficient parser to validate the grammatical structure of the C language subset utilizing syntactic analysis techniques.
- Implement a Syntax-Directed Translation (SDT) scheme to perform type checking, variable declaration verification, and other essential semantic validations concurrently with the parsing process.
- Detect lexical, syntactic, and semantic errors present in the source code.
- Build auxiliary structures such as symbol tables and hierarchical representations of the program.

2 Theoretical Framework

2.1 Compilers

A programming language is defined as a set of rules and notations that allow us to communicate with computers in order to develop software, applications, and systems. However, for a program to be executed, it is necessary to translate it into a format that the computer can understand and execute. The systems responsible for this process are known as compilers.

The compiler transforms high-level source code into low-level target code that the processor can execute directly. In general, this process can be understood as two blocks that perform different activities: analysis and synthesis.

In the analysis phase, the compiler acts as a "critical reader," whose objective is to understand the source code and verify that it is correct. It is divided into lexical analysis, syntax analysis, and semantic analysis. At the end of this phase, an intermediate representation is generated, which is a version of the code that is no longer text, but is still not machine language.

In the synthesis phase, the compiler is responsible for efficiency and the technical construction of the executable. It takes the validated intermediate representation and translates it into the target language. It mainly consists of intermediate code generation, code optimization, and target code generation.[1]

Compiler is divided into six phases: lexical analyzer, syntactic analyzer, semantic analyzer, intermediate code generation, code optimization, and target code generation.

2.2 Lexical Analysis

The first phase of compilation is called lexical analysis. In this phase, the analyzer takes the source code as input, reads its characters, and groups them into meaningful units called tokens, together with a terminal symbol used to make syntax analysis decisions. A token represents a lexical category within a programming language, such as:

- Keywords (int, float, return)
- Identifiers (x, a, b, c)
- Operators (=, +, -, *)
- Constants (2, 3, 5, 0)
- Punctuation marks ((,), {, }, ;)

- Literal ("Hello World")

An input sequence that forms a token is called a lexeme; therefore, the lexical analyzer isolates the syntax analyzer from token representation through lexemes. To perform this recognition, lexical analyzers use pattern identification mechanisms that allow each fragment of the input text to be classified. These patterns are generally described through regular expressions, which formally define how certain language elements can be formed.[1]

2.3 Context Free Grammar (CFG)

Once the token sequence has been obtained, the syntax analysis phase begins. However, before explaining this process, it is necessary to understand Context-Free Grammars (CFG), since they are the formal basis of syntax analysis. In general, a grammar can be defined using the quadruple:

$$G = (V, \Sigma, P, S)$$

Where:

- V - syntactic variables that denote sets of strings, known as non-terminals. The sets of strings denoted by non-terminals help define the language generated by the grammar. Non-terminals impose a hierarchical structure on the language, making them key for syntax analysis.
- Σ - basic symbols from which strings are formed, known as terminals. Terminals are the primary components of the tokens produced by the lexical analyzer.
- P - refers to the productions of a grammar. These specify how terminals and non-terminals can be combined to form strings.
- S - represents the start symbol. This symbol is a special non-terminal from which the derivation of strings in the language defined by the grammar begins.

CFGs help us specify the syntax of a programming language; each language has precise rules that prescribe the syntactic structure of well-formed programs. However, these rules constitute only a theoretical model. The transition to the implementation phase occurs when the syntax analyzer assumes the operational role.[1]

2.4 Syntax Analysis (Parser)

The syntax analyzer (Parser), once the token sequence is ready, is responsible for verifying that the sequence of token names can be generated by the grammar of the source language. The syntax analyzer organizes tokens into syntactic classes, also called non-terminals in the grammar, such as expressions, procedures, and other elements. If the sequence respects the established production rules, then it is considered syntactically valid, and the result is a parse tree, where the leaves represent the tokens and the internal nodes correspond to the syntactic classes, showing how the program

structure is organized according to the grammar. Otherwise, the compiler reports a syntax error.[1]

The general process of syntax analysis can be represented in the following phases:

- Token Reception: The lexical analyzer delivers the token sequence to the syntax analyzer.
- Verification: It checks whether the tokens match the grammar production rules.
- Parse Tree Construction: It represents the hierarchical structure of the program according to the grammatical rules.
- Error Generation: If it finds an invalid sequence, it generates an error message indicating the problem.

2.5 Parsing Methods

Parsing methods are classified into:

- Top-Down: This method derives the input starting from the start symbol and expanding productions until matching the received tokens; that is, it builds parse trees from the top (root) to the bottom (leaves). This same classification is divided into non-deterministic and deterministic methods. Non-deterministic parsers are those that, when facing a decision, do not know with certainty which path to take and must try alternatives (brute force). Deterministic parsers are those that can predict exactly which rule to use, based only on the current symbol. Among deterministic parsers are LL(1) parsers, which use one lookahead symbol to decide which production to apply. Top-Down syntax analysis is usually easier to implement manually, although it requires eliminating ambiguity, left recursion, and applying factoring when necessary.[1]
- Bottom-Up: This method works in the opposite way to Top-Down methods: it begins with the input tokens and attempts to reduce sequences until reaching the start symbol; that is, it builds parse trees from the leaves upward to the root. As with the previous method, Bottom-Up parsers can be classified as ambiguous and predictive. Ambiguous parsers may explore multiple reduction paths simultaneously in the presence of ambiguity. Predictive parsers use lookahead symbols to make deterministic decisions without needing backtracking; these include LR, SLR, LALR, and CLR parsers.
  These parsers are more powerful than Top-Down parsers, since they accept a wider variety of grammars and are widely used in automatic tools such as Yacc or Bison.[1]

2.6 FIRST Sets

FIRST sets determine which terminals may appear at the beginning of derivations. Formally:

$$\text{FIRST}(\alpha)$$

represents the set of terminal symbols that can start derived strings from α.

These sets are essential for:

- Calculating lookaheads.
- Constructing LR items
- Generating parsing tables.

2.7 LR(1) Items and Closure

An **LR(1) item** is a production with a dot ($\bullet$) indicating how much we have parsed, plus a lookahead terminal. Example:

$$E \to E \bullet + T \quad , \quad \text{lookahead} = \{\$, +,)\}$$

The Closure operation automatically expands elements when the period appears before a non-terminal, adding new productions needed to continue the syntactic analysis.[2]

2.8 GOTO and State Construction

The GOTO function allows shifting the dot over a symbol and constructing new states of the LR automaton.

Formally:

$$\text{goto}(I, X) = \text{closure}(\{A \to \alpha X \bullet \beta, a \mid A \to \alpha \bullet X\beta, a \in I\})$$

State construction is fundamental for generating the ACTION and GOTO tables used during parsing.[2]

2.9 Shift-Reduce Parsing

Shift-Reduce parsing simulates an LR automaton using stacks.
The main operations are:

- **Shift**: Shifts a token onto the stack.
- **Reduce**: Applies a grammar production.
- **Accept**: Indicates that the input is valid.
- **Error**: Indicates an invalid sequence.

The parser uses:

- A state stack
- A semantic stack for SDT (Syntax-Directed Translation).

2.10 Syntax-Directed Translation

Syntax-Directed Translation (SDT) allows executing semantic actions during the parsing process.
Each grammar production has associated actions that allow:

- Constructing abstract syntax trees,
- Checking types,
- Managing symbol tables,
- Evaluating expressions,
- Detecting semantic errors.

SDT uses semantic attributes:

Synthesized attributes
They flow from the children to the parent node.

Inherited attributes
They receive information from the parent or siblings.

2.11 Abstract Syntax Tree (AST)

The Abstract Syntax Tree (AST) represents the logical structure of the program, eliminating unnecessary elements such as parentheses or delimiters.[2]
The nodes implemented in the project include:

- Constants
- Identifiers
- Arithmetic operators
- Declarations

The AST facilitates the subsequent stages of validation and code generation.

2.12 Symbol Table

The symbol table stores information related to variables and program elements.
Among the stored data are:

- Name
- Type
- Value

- Scope

The table allows:

- Registering variables
- Verifying declarations
- Validating usages,
- Detecting redeclarations

2.13 Intermediate Code Generation

After the source code has been lexically, syntactically, and semantically validated, many compilers generate an intermediate representation of the program before producing machine code. This intermediate representation decouples the analysis and synthesis phases of the compiler, facilitating optimization and portability between architectures.

The generation of intermediate code simplifies subsequent stages such as optimization and target code generation, as it provides a representation that is easier to manipulate than the original source code.

2.14 Code Optimization

Code optimization is the phase responsible for improving the program's efficiency without altering its logical behavior. Its primary goal is to reduce execution time, memory consumption, or the size of the generated code.
Optimizations can be classified into:

- Machine-independent optimizations
- Machine-dependent optimizations

Among the most common techniques are:

- Dead code elimination
- Constant propagation
- Algebraic simplification
- Common sub expression elimination
- Loop optimization

2.15 Target Code Generation

Target code generation constitutes the final phase of the compiler. Its purpose is to translate the intermediate representation into specific instructions for a hardware architecture.
The target code can produce:

- Assembly code
- Machine code
- Bytecode
- Object files

During this phase, the compiler must consider aspects such as:

- Register management,
- Memory addressing,
- CPU instructions,
- Handling function calls.

Target code generation allows the program to execute directly on the target system.

2.16 Linker and Execution

After generating the target code, the linker comes into play. Its function consists of combining different object files, libraries, and external references to produce a final executable.
The linker resolves:

- Function references
- Global variables
- Memory addresses
- External dependencies

Once the linking is completed, the system generates an executable program capable of running directly on the corresponding operating system and processor.
Within the compiler's general workflow, this stage represents the final transition between source code analysis and the actual execution of the program.

3 Development

3.1 Grammar

```
      Program' → Program
       Program → TopLevelList
 TopLevelList → TopLevel | TopLevelList  TopLevel
      TopLevel → Statement | FunctionDecl

 StatementList → Statement | StatementList  Statement
     Statement → Declaration  ; | Assignment  ; | Block
   Declaration → TYPE  DeclList
      DeclList → DeclItem | DeclList  ,  DeclItem
      DeclItem → ID
                | ID  =  E
                | ID  [  E  ]
                | ID  [  E  ]  =  {  InitList  }
                | ID  [  E  ]  [  E  ]
                | ID  [  E  ]  [  E  ]  =  {  MatrixInitList  }
    Assignment → ID  =  E
                | ArrayAccess  =  E
                | MatrixAccess  =  E
         Block → {  StatementList  } | {  }

             E → OrExpr
       OrExpr → OrExpr  ||  AndExpr | AndExpr
      AndExpr → AndExpr  &&  EqExpr | EqExpr
       EqExpr → EqExpr  ==  RelExpr | EqExpr  !=  RelExpr | RelExpr
      RelExpr → RelExpr  <  AddExpr
                | RelExpr  >  AddExpr
                | RelExpr  <=  AddExpr
                | RelExpr  >=  AddExpr
                | AddExpr
      AddExpr → AddExpr  +  MulExpr | AddExpr  -  MulExpr | MulExpr
      MulExpr → MulExpr  *  UnaryExpr
                | MulExpr  /  UnaryExpr
                | MulExpr  %  UnaryExpr
                | UnaryExpr
```

UnaryExpr → `!` UnaryExpr | `-` UnaryExpr | `+` UnaryExpr | Primary

Primary → `(` E `)` | ID | CONST | FunctionCall | ArrayAccess | MatrixAccess

Statement → IfStatement

IfStatement → `if` `(` E `)` Block | `if` `(` E `)` Block `else` Block

Statement → WhileStatement

WhileStatement → `while` `(` E `)` EnterBreak EnterLoop Block ExitLoop ExitBreak

Statement → ForStatement

ForStatement → `for` `(` ForInit `;` E `;` ForUpdate `)`
EnterBreak EnterLoop Block ExitLoop ExitBreak

ForInit → ID `=` E | Declaration

ForUpdate → ID `=` E | ID `++` | ID `-`

Statement → SwitchStatement

SwitchStatement → `switch` `(` E `)` `{` EnterBreak CaseList `}` ExitBreak
| `switch` `(` E `)` `{` EnterBreak CaseList
DefaultItem `}` ExitBreak
| `switch` `(` E `)` `{` EnterBreak DefaultItem
`}` ExitBreak

CaseList → CaseItem | CaseList CaseItem

CaseItem → `case` CONST `:` Block

DefaultItem → `default` `:` Block

Statement → BreakStatement `;`

BreakStatement → `break`

EnterBreak → ϵ

ExitBreak → ϵ

Statement → ContinueStatement `;`

ContinueStatement → `continue`

EnterLoop → ϵ

ExitLoop → ϵ

FunctionDecl → FunctionHeader EnterFunction FunctionBody ExitFunction

FunctionHeader → TYPE ID `(` `)` | TYPE ID `(` ParamList `)`

ParamList → Param | ParamList `,` Param

Param → TYPE ID

FunctionBody → `{` EnterParams StatementList `}` | `{` EnterParams `}`

EnterFunction → ϵ

ExitFunction → ϵ

EnterParams → ϵ

Statement → ReturnStatement `;`

ReturnStatement → `return` | `return` E
Statement → FunctionCall `;`
FunctionCall → ID `(` `)` | ID `(` ArgList `)`
ArgList → E | ArgList `,` E
Statement → PrintStatement `;`
PrintStatement → `print` `(` E `)`
| `printf` `(` CONST `)`
| `printf` `(` CONST `,` ArgList `)`
ArrayAccess → ID `[` E `]`
MatrixAccess → ID `[` E `]` `[` E `]`
InitList → E | InitList `,` E
MatrixInitList → MatrixRow | MatrixInitList `,` MatrixRow
MatrixRow → `{` InitList `}`

3.2 Phase 1 —Lexical Analysis

3.2.1 Token Table Design

The token recognition table is defined as a prioritized ordered list of `(pattern, token_type)` pairs. Order is critical: patterns listed earlier take precedence over later ones. This is why multi-character operators such as `==`, `!=`, `<=`, `>=`, `++`, `-`, `+=`, `-=`, `&&`, and `||` are defined before their single-character counterparts (`=`, `<`, `>`, `+`, `-`, `&`, `|`). Without this ordering, the tokenizer would greedily match only the first character of a compound operator and produce two incorrect tokens instead of one correct one. All patterns are compiled into Python regex objects once at module load time. The compiled list is iterated for every token match.

Token categories defined in the table are:

- **Comments:** Both `//` single-line and `/* */` multi-line patterns, mapped to None so they are silently discarded without producing a token.
- **Literals:** Double-quoted strings and single-quoted characters, captured as the literal category including their surrounding delimiters.
- **Constants:** Numeric values, integers and floating-point numbers.
- **Compound operators:** The multi-character operators listed above.
- **Simple operators:** Single-character arithmetic, relational, logical, and assignment operators.
- **Punctuation:** The full set includes type keywords (`int`, `float`, `double`, `char`, `void`, `bool`, `long`, `short`, `unsigned`), control-flow keywords (`if`, `else`, `while`, `for`, `switch`, `case`, `default`, `break`, `continue`), function-related (`return`, `main`,

`void`), output (`print`, `printf`), and modifiers (`static`, `const`, `typedef`, `sizeof`, `struct`).

- **Identifiers** The pattern `[a-zA-Z_][a-zA-Z0-9_]*`, placed after all keyword patterns so that keywords are matched first.
- **Special characters:** The # preprocessor marker, captured as its own category.
- **Whitespace:** Mapped to `None` and discarded like comments.

3.2.2 Tokenization Algorithm

The tokenizer processes the source code line by line, maintaining both a line counter (for error messages) and a character position pointer within each line. For each position, it attempts to match each compiled pattern against the remaining substring using `re.match`, which anchors the match at the current position.

The first pattern that produces a match is accepted. The matched lexeme is consumed (the position advances by the length of the match), and if the token type is not `None`, a four-element tuple `(type, value, line, column)` is appended to the token list. This tuple design is what propagates positional information through all later phases, enabling precise error messages.

If no pattern matches the current character, the tokenizer reports an error identifying the exact line and column of the invalid symbol and immediately returns `None` to signal failure to the caller, preventing any downstream phases from running on partial or malformed input.

Two public functions wrap the tokenizer, One that reads source text from a file path (with file-not-found handling), and one that accepts a string directly, enabling both the file-based terminal interface and the GUI's live code editor.

3.3 Phase 2 — Syntactic Analysis

3.3.1 Grammar Design and Operator Precedence

The grammar is encoded as an explicit Python list of `(non-terminal, [symbols])` production tuples. The language does not use any external parser-generator tool — the grammar, table construction, and driver loop are all implemented from scratch.

Operator precedence is encoded structurally in the grammar through a hierarchy of non-terminals, from lowest to highest binding power:

$$\text{E} \rightarrow \text{OrExpr} \rightarrow \text{AndExpr} \rightarrow \text{EqExpr} \rightarrow \text{RelExpr} \rightarrow \text{AddExpr} \rightarrow \text{MulExpr} \rightarrow \text{UnaryExpr} \rightarrow \text{Primary}$$

Each level only refers to the level immediately above it (tighter binding), so the grammar's own structure forces the parser to build deeper (higher-precedence) subtrees first. This eliminates the need for external precedence declarations.

The full grammar covers:

- **Top-level structure:** A `Program` consists of a `TopLevelList` of `TopLevel` items, each of which is either a `Statement` or a `FunctionDecl`. This allows both global variable declarations and function definitions at file scope.
- **Declarations:** The `Declaration` non-terminal handles `TYPE DeclList` where `DeclList` is a comma-separated list of `DeclItem` entries. Each `DeclItem` can be a bare identifier (`int x`), an initialized identifier (`int x = expr`), an array (`int arr[size]`), or an initialized array (`int arr[3] = {1, 2, 3}`). This single production covers all scalar and array declaration forms.
- **Assignments:** Handled separately from declarations since an already-declared variable being assigned does not carry a type prefix. Both scalar assignments and array-element assignments (`arr[i] = expr`) are covered.
- **Blocks and scoping:** A `Block` is either `{ StatementList }` or `{ }` for empty blocks. Scope entry and exit are triggered directly by the parser when it shifts `{` and `}` tokens respectively, before any semantic action runs.
- **Control flow:** `if/else`, `while`, `for`, `switch/case/default`, `break`, and `continue` are each given dedicated non-terminals, allowing semantic markers to be placed at grammatically precise positions.
- **Semantic marker non-terminals:** `EnterBreak`, `ExitBreak`, `EnterLoop`, `ExitLoop`, `EnterFunction`, `ExitFunction`, and `EnterParams` are epsilon productions (productions with an empty right-hand side). They produce no tokens and therefore cause an immediate reduction, triggering their semantic action at exactly the point in the parse where the corresponding context change must occur — for example, `EnterBreak` fires just before the body of a `while` or `for` loop so that any break encountered inside is valid.
- **Functions:** `FunctionDecl` is structured as `FunctionHeader EnterFunction FunctionBody ExitFunction`, allowing the semantic analyzer to register the function's signature before processing its body, and to clean up function context state after the body is fully reduced.
- **Expressions:** The full binary expression hierarchy, unary operators (`!`, `-`, `+`), function calls, array accesses, identifiers, and constants are all expressed as `Primary` variants.
- **Print/printf:** Treated as dedicated statement forms rather than ordinary function calls so their format-string validation logic can be applied directly by the SDT.

3.3.2 FIRST Set Computation

Before building the LR(1) automaton, FIRST sets are computed for all grammar symbols using the standard iterative fixed-point algorithm. A FIRST set for a symbol X is the set of terminals that can begin any string derived from X. For terminal symbols the FIRST set is the symbol itself. For non-terminals, the algorithm iterates over all productions and propagates: if a production's body starts with a non-terminal whose FIRST set is known, those terminals are added to the left-hand side's FIRST set. The ε (epsilon) marker is propagated when all symbols in a production's body are nullable. The algorithm repeats until no new additions occur (fixed point).

The function `primero_de_cadena` extends this to sequences of symbols, which is needed during LR(1) closure computation to determine lookaheads for new items.

3.3.3 LR(1) Item Set Construction

An LR(1) item is a four-tuple `(lhs, rhs_tuple, dot_position, lookahead)`. The dot position marks how far into the production the parser has already recognized; the lookahead is the terminal that must appear next after reducing this item.

The closure of a set of items is computed by repeatedly adding new items: for every item whose dot precedes a non-terminal B, all productions for B are added with dot at position 0, and their lookaheads are computed from the FIRST set of the remaining symbols after B plus the original item's lookahead.

The `goto` function computes the set of items reachable from a given item set by shifting a specific symbol — it collects all items whose dot is before that symbol, advances the dot, and computes the closure of the result.

The full canonical LR(1) automaton is built by breadth-first exploration: starting from the initial item $(\text{Program}' \to \bullet\text{Program}, \$)$, `goto` is computed for every possible symbol (terminal and non-terminal) from every state, and new states are added to the worklist if they haven't been seen before. State identity is based on frozenset equality of the full item set including lookaheads.

3.3.4 LALR(1) State Merging

The canonical LR(1) collection is then merged into LALR(1) states. Two LR(1) states can be merged if they have identical cores — that is, if they contain the exact same set of `(lhs, rhs, dot_position)` triples regardless of lookaheads. Merging combines the lookahead sets of matching items from both states. This dramatically reduces the number of states (from hundreds to a manageable size) while preserving the practical parsing power needed for the grammar.

A mapping is maintained from every original LR(1) state index to its corresponding LALR(1) state index. This mapping is then used to translate all transition targets in the ACTION and GOTO tables from LR(1) state numbers to LALR(1) state numbers.

3.3.5 ACTION and GOTO Table Construction

The ACTION table maps `(state, terminal)` pairs to parser actions:

- **Shift** — when the dot is before a terminal a and a transition exists, the action is `S<next_state>`.
- **Reduce** — when the dot is at the end of a production, the action is `R<production_number>` for each lookahead terminal.
- **Accept** — when the item is (Program$'$ $\rightarrow$ Program $\bullet$, \$), the action is `acc`.

The GOTO table maps `(state, non-terminal)` pairs to the next state after a reduction pushes the non-terminal.

Any `(state, terminal)` cell that would receive two different actions (shift vs. reduce, or reduce vs. reduce) is a grammar conflict. The implementation detects this and raises an exception immediately, making it impossible to produce an ambiguous parser silently. The grammar was designed to be conflict-free under LALR(1).

3.3.6 Token-to-Grammar-Symbol Mapping

Before the parser driver runs, the raw token stream from the lexer (which uses categories like keyword, identifier, operator, constant) must be remapped to the terminal symbols the grammar expects. This mapping handles several non-trivial cases:

- All type keywords (`int`, `float`, `double`, `char`, `void`, `bool`, `long`, `short`, `unsigned`) are unified into the single terminal `TYPE`. This allows the grammar to use one production for all declarations without listing every type keyword.
- The keyword `main` is remapped to `ID` rather than treated as a keyword, so function definitions like `int main()` parse correctly through the `FunctionDecl` production.
- The identifiers `true` and `false` are remapped to `CONST` since they are boolean constant literals, not variable names.
- String literals and numeric constants both map to `CONST`.
- Compound and simple operators, and all punctuation tokens, are mapped to their literal string form (e.g., `"=="`, `"+"`, `";"`) since those are the exact terminal symbols used in the grammar.

Any token that does not match a known mapping raises an exception immediately, preventing the parser from running with an unknown symbol.

3.3.7 Shift-Reduce Driver Loop

The parser maintains two parallel stacks: a state stack and a semantic-value stack. The state stack drives table lookups; the semantic-value stack accumulates the values that semantic actions will consume.

On each iteration the driver reads the current state from the top of the state stack and the current input terminal, and looks up the action in the ACTION table:

- **On Shift(n)** — the current lexeme is pushed onto the semantic stack, state n is pushed onto the state stack, and the input position advances.
- **On Reduce(k)** — production k has right-hand side of length m. The top m elements are popped from both stacks. The semantic action for production k is called with those m values and their source positions. Its result is pushed onto the semantic stack. The GOTO table is consulted with the now-exposed state and the production's left-hand side to find the next state, which is pushed onto the state stack.
- **On Accept** — parsing is complete. The single remaining value on the semantic stack is the root of the AST.
- **On None (no entry in ACTION)** — a syntax error is reported with the line and column of the current token and the list of terminals that would have been valid in this state (the keys of the state's ACTION entry).

Semantic action exceptions are caught individually during reductions so that a single SDT error does not abort the entire parse. The error is recorded and parsing continues, collecting all semantic errors before reporting them at the end.

3.4 Phase 3 — Syntax-Directed Translation and AST Construction

3.4.1 AST Node Structure

Every meaningful grammar construct reduces to a `Nodo` (Node) object. Each node stores: a `tipo` (type string identifying the construct, e.g., `'DECL'`, `'ASSIGN'`, `'+'`, `'IF'`, `'CALL'`), an optional `valor` (the node's primary scalar payload, such as a variable name, constant value, or operator), a list of `hijos` (child nodes), and the source `linea` and `columna` where the construct began. Leaf nodes (identifiers, constants) carry their value directly; operator nodes carry their children as operands; statement nodes carry their sub-expressions and blocks as children.

Two auxiliary value types exist alongside plain Python values: `ValorString` wraps string literals to distinguish them from plain Python `str` values used for identifiers; `ValorDesconocido` (unknown value) represents values whose concrete content cannot be determined at SDT time — for example, parameters inside a function body or return values from called functions — but whose type is known. This distinction is critical: many semantic checks (type compatibility, operator applicability) can still be performed on unknown values using only type information.

3.4.2 Symbol Table with Nested Scopes

The symbol table is implemented as a stack of scope objects (`ambito_pila`). Each scope is an independent dictionary mapping variable names to their type and value. When the parser shifts a `{` token, a new scope is pushed; when it shifts `}`, the top scope is popped and all variables declared in it are discarded.

Variable lookup (`buscar_variable`) searches the scope stack from top (innermost) to bottom (global scope), implementing standard lexical scoping. Declaration always targets the current (innermost) scope. This correctly handles shadowing: a variable declared in an inner block can share a name with one in an outer block without conflict.

Operations on the symbol table:

- `declarar` — registers a new variable with its type. Raises a semantic error if the name already exists in the current scope (not outer scopes). Also rejects `void` as a variable type.
- `asignar` — stores a value into an already-declared variable, first passing the value through the type conversion system to enforce type compatibility.
- `obtener` — retrieves a variable's type and value, raising a semantic error if the name is not found anywhere in the scope chain.
- `declarar_array` — registers an array variable, storing its element type, size, and a pre-allocated list of `None` values. Validates that the size is a positive integer known at SDT time.
- `asignar_array` — writes a value to a specific element, validating that the index is within bounds and that the value is type-compatible with the array's element type.
- `obtener_array` — reads an element, raising an error if the index is out of bounds or if the element has not yet been initialized (value is still `None`).

3.4.3 Function Table

A separate `TablaFunciones` object records all declared functions. Each entry stores the function's return type and its parameter list (each parameter as a `{tipo, nombre}` dictionary). Registration happens when the `FunctionHeader` production reduces — at this point the function's signature is fully known and can be inserted into the table before the body is processed. This is what allows recursive functions to call themselves.

Validation at declaration time: duplicate function names raise an error; `void` parameters are rejected; duplicate parameter names within one function are rejected.

Validation at call sites: the argument count is compared against the registered parameter count, and each argument's evaluated type is checked for compatibility with the corresponding parameter's type using the type conversion system.

3.4.4 Type System and Coercion

The type system defines two numeric families: integer types (`int`, `long`, `short`, `unsigned`) and real types (`float`, `double`), plus `char`, `bool`, and `void`.

`convertir_a_tipo` is the central coercion function. It accepts a value and a target type and either returns the correctly converted value or raises a semantic error. Its rules are:

- `bool`: any numeric or character value is accepted and converted via Python's truthiness.
- `char`: only integer values in the range 0–255 (or single-character strings) are accepted; non-integer floats are rejected.
- **Integer types** (`int`, `short`, `long`, `unsigned`): floats with fractional parts are rejected; character values are coerced via their ASCII code; the result is range-checked against the type's defined minimum and maximum.
- **Real types** (`float`, `double`): any numeric or character value is widened to float.
- `void`: only assignable to itself.

When the input value is a `ValorDesconocido`, the function checks only whether the source and destination types are compatible (using `tipos_convertibles`) and returns a new `ValorDesconocido` with the destination type, propagating the unknown through the expression tree without raising a false error.

`tipo_aritmetico` determines the result type of a binary arithmetic operation using standard C promotion rules: if either operand is `double`, the result is `double`; else if either is `float`, the result is `float`; otherwise the result is `int`.

3.4.5 Expression Evaluation

The evaluator `evaluar_con_tipo` walks an AST node recursively and returns both the computed value and its type. Key cases:

- `CONST` node — the stored value is returned directly; its type is inferred from its Python type (`bool`, `int`, `float`) or by examining string content (single-character strings become `char`; double-quoted strings become `string`).
- `ID` node — the variable is looked up in the scope chain. Using an uninitialized variable (value is `None`) raises a semantic error. Using an array name without an index subscript also raises an error.
- `ARRAY_ACCESS` node — the index expression is evaluated, validated as integer-compatible, and used to retrieve the element from the symbol table.

- `CALL` node — the return type is fetched from the function table. A `void` function used as an expression operand raises an error. Since the actual return value is unknown at SDT time, a `ValorDesconocido` with the correct return type is returned.

- **Unary operators** (`!`, `NEG`, `POS`) — `!` applies logical negation and always returns `bool`; `NEG` and `POS` require numeric or character operands, apply arithmetic negation or identity, and respect the type promotion rules.

- **Logical operators** (`&&`, `||`) — both operands are evaluated; if either is unknown, an unknown `bool` is propagated; otherwise the result is the short-circuit-equivalent boolean computation.

- **Relational and equality operators** — both operands are reduced to numeric values via `valor_numerico`; the comparison is performed and a `bool` result is returned.

- **Arithmetic operators** (`+`, `-`, `*`, `/`, `%`) — both operands are evaluated; the result type is determined by `tipo_aritmetico`. Division by a known zero raises a semantic error. Modulo rejects real operands. Integer division truncates toward zero, matching C semantics.

3.4.6 Control Flow Semantic Checks

- **break and continue context tracking** — two global counters, `break_contexto` and `loop_contexto`, track nesting depth. The epsilon production `EnterBreak` fires on reducing just before any loop or switch body and increments `break_contexto`; `ExitBreak` decrements it. Similarly, `EnterLoop` and `ExitLoop` manage `loop_contexto` for continue. When a break statement is reduced, `validar_break` checks that `break_contexto` > 0; when continue is reduced, `validar_continue` checks `loop_contexto` > 0. Using either keyword outside the appropriate construct raises a semantic error.

- **if / else** — the condition expression is evaluated by `_validar_condicion`, which checks that the condition type is not void and that the expression is free of errors (undeclared variables, division by zero, etc.). The branch bodies are not executed — they are only structurally assembled as AST children. This means the SDT does not track which branch was taken; it validates both branches syntactically and semantically but treats the result as a control-flow construct, not a value-producing expression.

- **while** — the condition is validated (same check as if). The loop body is assembled but not iterated. Because the iteration count is unknown at SDT time, any variables modified inside the loop body that feed back into expressions produce `ValorDesconocido` values if needed.

- **for** — the initializer (`ForInit`) can be either a declaration or a simple assignment; in either case it is processed normally and the variable is added to the current scope. The condition and update expressions are validated for type correctness. The loop body follows the same treatment as while.

- **switch/case** — the switch expression is evaluated. Each case constant is type-checked for compatibility with the switch expression's type using `convertir_a_tipo`. Duplicate case values (after type conversion) are detected and rejected. A `default` clause is optional and accepts any block. The `EnterBreak/ExitBreak` markers wrap the entire switch body so that break is valid inside it.

3.4.7 Function Scope Management

When the `EnterFunction` epsilon production reduces, the system takes a snapshot of the entire current scope stack (`_snapshot_ambitos`). This snapshot is pushed onto `snapshots_funcion`. The function's header information (name, return type, parameters) is stored in `funcion_contexto_pila` so that return-statement validation knows which function is currently being analyzed.

When `EnterParams` reduces (just inside the opening `{` of the function body), `declarar_parametros_funcion_actual` iterates the function's parameter list and declares each parameter in the current scope with a `ValorDesconocido` value. This makes parameters visible to the body's expressions while correctly marking their values as unknown (since actual arguments are not available at SDT time).

When `ExitFunction` reduces, the scope stack is restored from the saved snapshot. This ensures that variables declared inside a function body do not leak into the outer scope and that the outer scope's variable values are not corrupted by anything the function body computed.

Return-statement validation (`validar_return`) checks: that return appears within a function (context stack is non-empty); that a void function does not return a value; that a non-void function does return a value; and that the returned expression's type is compatible with the declared return type. The function header entry is also updated with a `tiene_return` flag, which `validar_funcion_retorno` later checks to confirm that non-void functions have at least one return path.

3.4.8 Printf Format String Validation

`printf` receives special treatment beyond a generic function call. The first argument must be a string literal (type `string`); this is verified immediately. The format string is then scanned for `%` specifiers. Supported specifiers are `%d/%i` (integer-compatible), `%u` (unsigned integer-compatible), `%f` (numeric), `%c` (character or integer), `%s` (string), and `%b` (bool). The `%%` escape is recognized and does not consume an argument. Any unrecognized specifier raises a semantic error. The count of specifiers must exactly match the count of additional arguments, and each argument's type must be compatible with its corresponding specifier.

3.5 AST Export and Visualization

After a fully successful parse and SDT pass, the AST root is printed to the console as an indented text tree (depth-first, two spaces per level), displaying each node's type and value.

The AST is also exported to Graphviz DOT format. The exporter performs a recursive depth-first traversal, assigning each node a unique ID (`n0`, `n1`, ...). For each node it emits a `[label="..."]` definition using `shape=box`, `style="rounded"`, and for each child it emits a directed edge. Special characters in node labels (`"`, `\`, newlines) are escaped to produce valid DOT syntax.

The DOT file is then passed to the dot command-line tool (if Graphviz is installed) with `-Tpng` to render a PNG image. This image is loaded by the GUI into a scrollable canvas widget. If Graphviz is not available, only the `.dot` file is produced and the GUI shows a placeholder.

3.6 Entry Points and User Interfaces

3.6.1 Terminal Interface

The terminal interface presents a simple input loop asking the user to choose between archive (file path input) and terminal (direct string input). In either case the source is passed to the lexer, and on success the token list is passed to the parser. The loop repeats on invalid selections and exits cleanly on exit.

3.6.2 Graphical Interface

The GUI is built with Tkinter and organized into a horizontal `PanedWindow` with resizable panes. The left pane contains a `ScrolledText` code editor pre-populated with a sample expression. Two buttons trigger compilation and clear all panels respectively.

The right pane is a `ttk.Notebook` with four tabs:

- **Output** — streams all compilation phase messages including lexer status, parser result, symbol table dump, function table dump, and any error details. A status label at the bottom shows a color-coded summary (green for success, red for errors).
- **Symbol Table** — a `ttk.Treeview` table populated after successful compilation, showing variable name, type, and current value for all entries in the global symbol table.
- **AST (Text)** — displays the indented text-form of the AST extracted from the parser output.
- **AST (Image)** — renders the Graphviz-generated PNG in a scrollable canvas. The canvas handles both vertical and horizontal scroll, and the image is centered when it is smaller than the canvas area.

Compilation is executed by calling the lexer's `analizeterminal` function on the editor content, then passing the token list to `parser.analizar`. Standard output is captured with `contextlib.redirect_stdout` so that all print statements from the parser and SDT modules are redirected to the GUI's output tab rather than the terminal.

3.6.3 Dependency Checker

At startup the GUI calls `check_all`, which independently verifies three dependencies: Tkinter (via import), Pillow (via import of `PIL.Image`), and Graphviz (by running `dot -V` as a subprocess and checking the return code). Tkinter is mandatory — its absence triggers an error dialog and immediate exit. Pillow and Graphviz are optional; their absence triggers a warning dialog listing the missing packages and their installation commands, after which the application continues with reduced functionality (no AST image rendering).

3.7 Test Cases

3.7.1 Test 1 — Valid Declarations, Arrays, Indexes, and Functions (05_function_param_index_ok.c)

Input: Definition of function `pick` taking an integer index, initializing a local 3-element array, and returning the element, followed by `main` which calls `pick(1)`, prints, and returns the result.

- **Lexer output:** 32 tokens — six keywords (`int`, `return`), six identifiers (`pick`, `index`, `data`, `main`, `selected`, `print`), seven punctuation symbols (`(`, `)`, `{`, `}`, `[`, `]`, `;`), four constants (`3`, `8`, `13`, `21`), and operators (`=`, `,`). No errors.
- **Parser output:** Parsing success. The input matches the Program → FunctionList → FunctionDefinition chain for both `pick` and `main`. The Parse Tree successfully groups the array declaration, array indexing, and function calls. Two `FUNC_DEF` nodes, two `DECL` nodes, and two `RETURN` nodes are built in the AST.
- **SDT output:** `pick` defined as function (`int` → `int`) with parameter `index`; local array `data` allocated with size 3 and initialized with `{8, 13, 21}`; returns element at `data[index]`. `main` defined as function (() → `int`); `selected` declared as `int`, assigned the result of `pick(1)` (evaluates to 13); `print(13)` executed. Symbol table manages global function scopes and local variable offsets. AST and DOT file generated successfully.

3.7.2 Test 2 — Lexical error (test_err_1.c)

Input: Function `main` containing an assignment with an unrecognized symbol `@`.

- **Lexer output:** Lexical error. Unrecognized character `@` detected at line 2, column 16. Tokenization halted.

- **Parser output:** Parsing failed. No syntax tree could be constructed due to early lexer failure / unexpected token stream.
- **SDT output:** Compilation aborted. Semantic analysis not executed due to parsing failure. Symbol table remains empty for local scope, and no AST or DOT files were generated.

4 Results

This section will present the results obtained from testing the program using the graphical user interface.

4.1 GUI Execution

- `executable.bat`: for Windows.
- `executable.command`: for macOS and Linux.

These files internally call the setup scripts located in the `setup/` directory. These scripts are responsible for validating the system, checking dependencies, creating the virtual environment, installing Python packages, running the environment checker, and launching the compiler's graphical interface.

4.1.1 Expected Project Structure

The executable files must be located in the project's root directory, at the same level as the `src/`, `tools/`, and `setup/` directories.

The expected structure is as follows:

```
path/compilers/g5/05
|---- doc/
|---- outputs/
|---- pre/
|---- scripts/
|---- setup/
|---- setup_and_run.sh
|---- setup_and_run.ps1
|---- src/
|---- assets/
|---- backend/
|---- lexer/
|---- outputs/
|---- parser_sdt/
|---- tests/
|---- tools/
|---- check_environments.py
```

```
|---- executable.bat
|---- executable.command
|---- requirements.txt
```

It is important not to move the executables to another directory, as they are configured to locate the setup scripts relative to the project root.

4.1.2 Downloading the Project

The project can be obtained either by cloning the repository or downloading it as a compressed archive.

If Git is used, the procedure is:

```
git clone https://github.com/YukiGab/compiler.git
```

After cloning or extracting the project, the user should open the project's main directory using the operating system's file explorer.

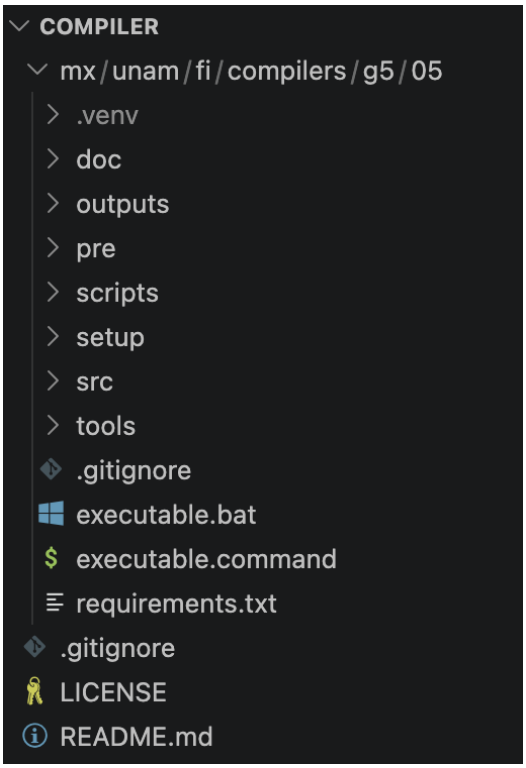

Figure 1: Project root directory showing the files `executable.bat`, `executable.command`, `src/`, `setup/`, `tools/`, and `requirements.txt`.

This screenshot is useful because it allows users to visually verify that the files are located correctly before running the project.

4.1.3 Running on Windows

On Windows, users must open the project's root directory and double-click the file:

`executable.bat`

This file automatically opens PowerShell and executes the Windows setup script using a temporary execution policy. This prevents users from having to manually type commands such as:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

The execution policy modification applies only during that session and does not permanently alter the system configuration.

When executed, the file performs the following actions:

1. Locates the project root directory.
2. Runs the script `setup/setup_and_run.ps1`.
3. Checks whether Python 3.10 or later is installed.
4. Checks whether Graphviz is installed and whether the `dot` command is available.
5. Creates or reuses the `.venv` virtual environment.
6. Installs the dependencies listed in `requirements.txt`.
7. Executes `tools/check_environment.py`.
8. If everything is correct, launches `src/GUI.py`.

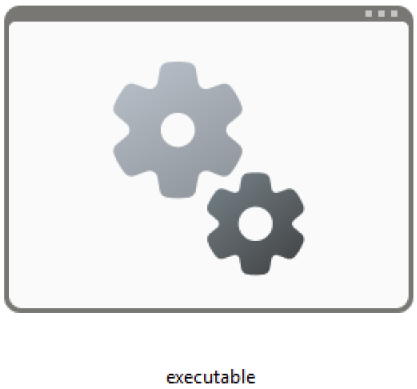

Figure 2: Running `executable.bat` from the project's root directory on Windows.

If any dependency is missing, the script will display instructions in the terminal. For example, it may indicate that Python or Graphviz must be installed, or that PowerShell should be run with administrator privileges. In that case, the user should follow the displayed instructions, close the window if necessary, and run `executable.bat` again.

4.1.4 Running on macOS and Linux

On macOS and Linux, users must open the project's root directory and execute:

```
executable.command
```

On macOS, this file can be opened by double-clicking it in Finder. On Linux, depending on the file manager and system configuration, it can be opened either by double-clicking or by selecting the option to run it as a program.

If the system indicates that the file does not have execution permissions, permission must be granted once. This can be done through the file properties in the file explorer by enabling execution permissions. It can also be done from a terminal using:

```
chmod +x executable.command
```

Once executed, the file internally calls:

```
setup/setup_and_run.sh
```

This script detects whether the operating system is macOS or Linux and performs the following actions:

1. Locates the project root directory.
2. Checks whether Python 3.10 or later is installed.
3. Checks whether Tkinter is available.
4. Checks whether Graphviz is installed and whether the `dot` command is available.
5. Creates or reuses the `.venv` virtual environment.
6. Installs the dependencies listed in `requirements.txt`.
7. Executes `tools/check_environment.py`.
8. If everything is correct, launches `src/GUI.py`.

Figure 3: Running `executable.command` from Finder on macOS.

On Linux, an equivalent screenshot may show the `executable.command` file within the file manager or the terminal window that opens during execution.

4.1.5 Permissions and System Dependencies

Although the executables automate most of the process, some dependencies may require special operating system permissions. This depends on the system configuration.

On Windows, if Python or Graphviz is not installed, the script may display installation commands using `winget`. If those commands require administrator privileges, a message will instruct the user to open PowerShell as an administrator and execute the indicated commands.

On macOS, if Homebrew, Python, or Graphviz is missing, the script will display installation instructions. In some cases, Homebrew must be installed before proceeding.

On Linux, if system dependencies are missing, the script may suggest commands using `sudo`, for example to install Python, Tkinter, or Graphviz. If a password prompt appears, it refers to the password of a user with administrative privileges.

The purpose of these scripts is to avoid silent or confusing installations. Therefore, whenever elevated permissions are required, the process informs the user before continuing.

4.2 Expected Output

If everything is configured correctly, the terminal should display messages similar to the following:

```
[INFO] Project detected at: .../PENTA-Compiler
[INFO] Operating system detected: ...
[OK] Python detected: Python 3.x.x
[OK] Graphviz detected: dot - graphviz version ...
[OK] Virtual environment .venv already exists.
[INFO] Installing Python dependencies...
[INFO] Running environment checker...
[OK] Environment ready. Launching PENTA Compiler...
```

After this output, the main PENTA Compiler window should open.

```
Iniciando el script de setup...
[INFO] Proyecto detectado en: /Users/olveratrech/compilers/compiler/compiler/mx/unam/fi/compilers/g5/05
[INFO] Sistema detectado: macos
[OK] Dependencias del sistema disponibles.
[OK] Python detectado: Python 3.12.3
[OK] Graphviz detectado: dot - graphviz version 14.1.5 (20260411.2331)
[OK] Entorno virtual .venv ya existe.
[INFO] Activando entorno virtual...
[INFO] Instalando dependencias de Python...
Requirement already satisfied: pip in ./.venv/lib/python3.12/site-packages (26.1.1)
Requirement already satisfied: customtkinter>=5.2.2 in ./.venv/lib/python3.12/site-packages (from -r requirements.txt (line 1)) (5.2.2)
Requirement already satisfied: Pillow>=10.0.0 in ./.venv/lib/python3.12/site-packages (from -r requirements.txt (line 2)) (12.2.0)
Requirement already satisfied: darkdetect in ./.venv/lib/python3.12/site-packages (from customtkinter>=5.2.2->-r requirements.txt (line 1)) (0.8.0)
Requirement already satisfied: packaging in ./.venv/lib/python3.12/site-packages (from customtkinter>=5.2.2->-r requirements.txt (line 1)) (26.2)
[INFO] Ejecutando verificador de entorno...

============================================================
PENTA COMPILER - ENVIRONMENT CHECK
============================================================
Platform : macOS-26.4.1-arm64-arm-64bit
Executable: /Users/olveratrech/compilers/compiler/compiler/mx/unam/fi/compilers/g5/05/.venv/bin/python
------------------------------------------------------------
[OK] python         Python 3.12.3
[OK] tkinter        Tkinter 8.6
[OK] customtkinter  CustomTkinter 5.2.2
[OK] pillow         Pillow 12.2.0
[OK] graphviz       dot - graphviz version 14.1.5 (20260411.2331) (/opt/homebrew/bin/dot)
============================================================
```

Figure 4: Expected terminal output when the environment is successfully validated.

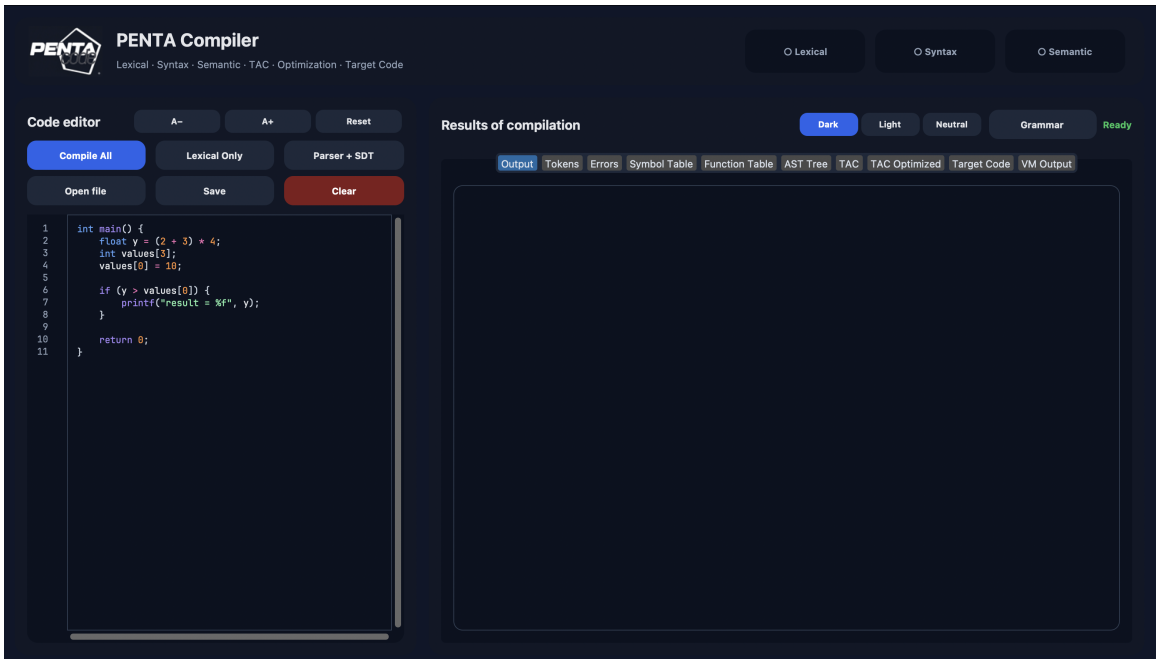

Figure 5: PENTA Compiler graphical interface opened after running the automated executable.

4.2.1 What to Do If an Error Occurs

If the process stops, the message displayed in the terminal should be read carefully. The most common cases are:

- **The file does not execute**: verify that it is being run from the project root directory and that the `setup/` directory exists.
- **Python is missing**: install Python 3.10 or later and run the file again.

- **Graphviz is missing**: install Graphviz and verify that the `dot` command is available.
- **Tkinter is missing**: install the package corresponding to the operating system.
- **Insufficient permissions**: follow the instructions displayed by the script in the terminal.
- **PowerShell blocks scripts**: run `executable.bat` again, as it applies a temporary execution policy to avoid this restriction.
- `executable.command` **does not open**: grant execution permissions to the file and try again.

After correcting the indicated issue, the corresponding executable should be run again.

4.2.2 Execution Summary

The execution workflow for the end user is simplified as follows:

- On Windows: double-click `executable.bat`.
- On macOS or Linux: execute `executable.command`.

4.3 Valid Test Cases

4.3.1 Test Case 1: Valid Declaration, Arrays, Arithmetic Operations and Control Structures

Figure 1 shows the source code input, which consists of variable declarations, arrays, arithmetic operations, and control structures. It also displays the available options for processing the code with our program (compile, lexical analyzer, syntactic analyzer, load files, and save).
Figure 2 shows the validation of its execution through all the compiler phases, as well as the output generated in the VM. The other images (Figure 3, 4, 5, 6, 7, 8, 9, 10, 11) show the output after passing through each compiler phase.

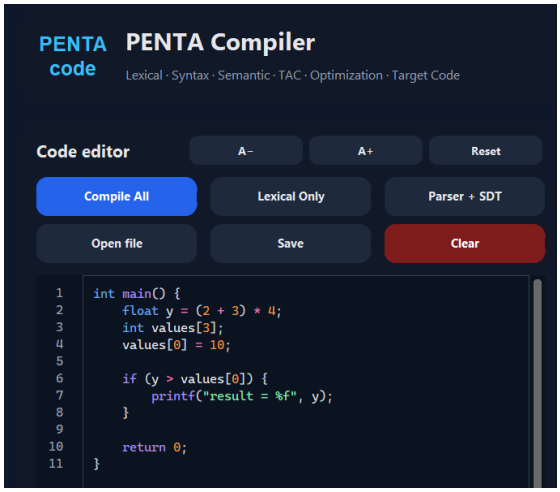

Figure 6: Input for Test Case 1

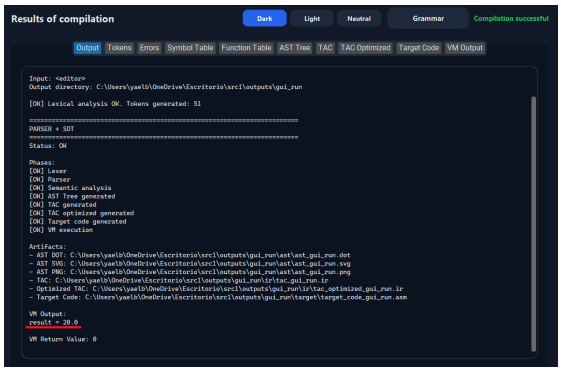

Figure 7: Output for Test Case 1

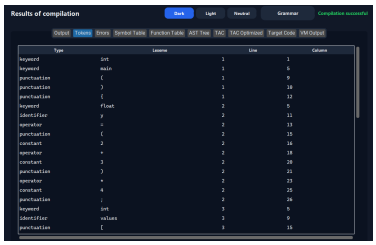

Figure 8: Tokens

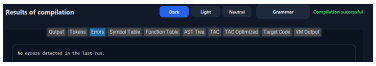

Figure 9: Errors

Figure 10: Symbol Table

Figure 11: Function Table

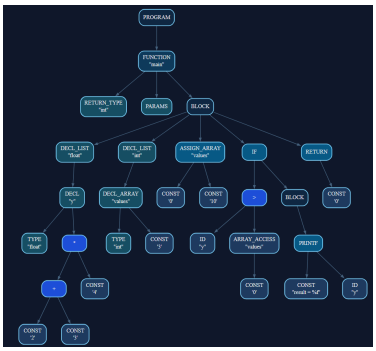

Figure 12: AST Tree

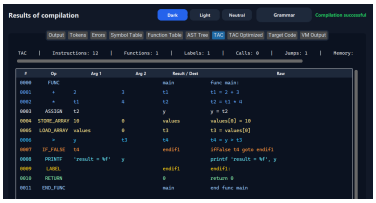

Figure 13: TAC

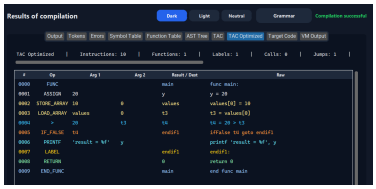

Figure 14: TAC Optimized

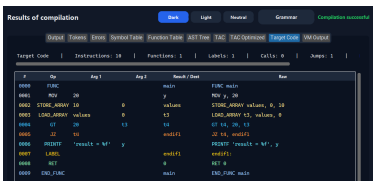

Figure 15: Target Code

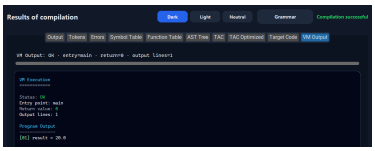

Figure 16: VM Output

4.3.2 Test Case 2: Valid Declarations, Arrays, Indexes, and Functions (05_function_param_index_ok.c)

Figure 12 shows the input source code, which consists of variable declarations, arrays, indices, functions, and parameters. It also shows the available options for processing the code with our program (compile, lexical analyzer, parser, load files, and save). Figure 13 shows the validation of its execution through all the compiler phases, as well as the output generated in the virtual machine. The remaining figures (14, 15, 16, 17, 18, 19, 20, 21, and 22) show the output after passing through each compiler phase.

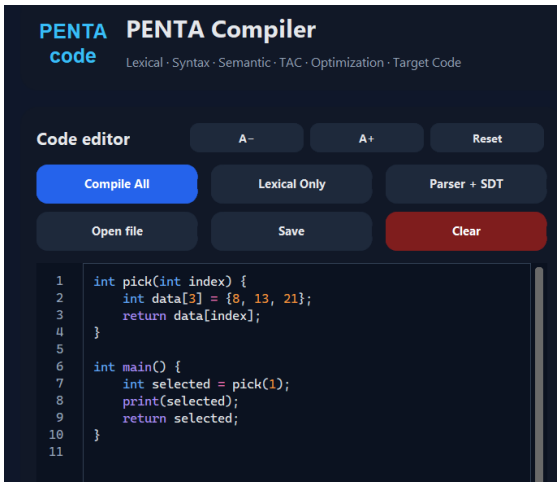

Figure 17: Input for Test Case 2

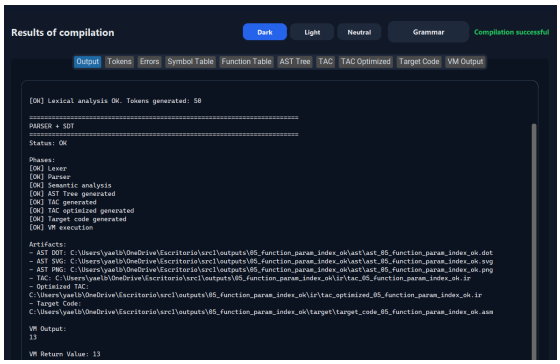

Figure 18: Output for Test Case 2

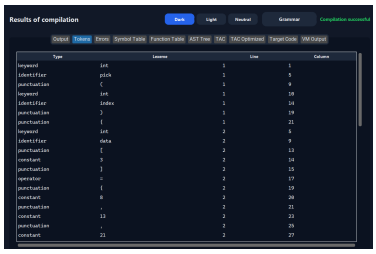

Figure 19: Tokens

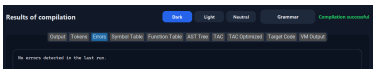

Figure 20: Errors

Figure 21: Symbol Table

Figure 22: Function Table

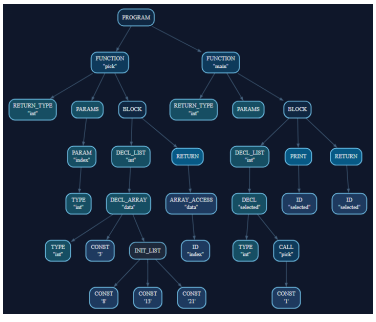

Figure 23: AST Tree

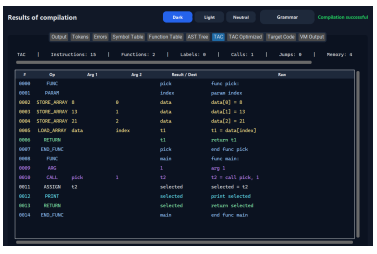

Figure 24: TAC

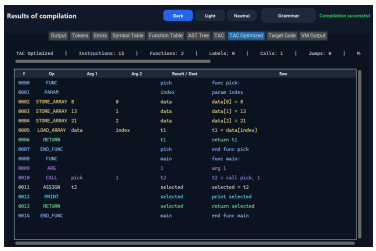

Figure 25: TAC Optimized

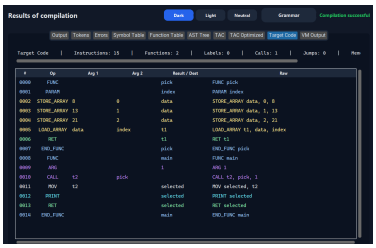

Figure 26: Target Code

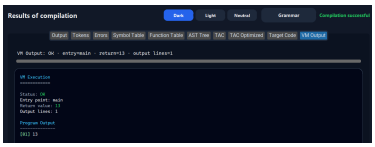

Figure 27: VM Output

4.3.3 Test Case 3: Full Language Feature Integration (05_full_feature_succes.c)

Figure 23 shows the input source code utilized for validating the compiler implementation. The test program integrates multiple language features, including variable declarations, arrays, function calls, arithmetic and logical operations, conditional and iterative control structures, semantic validations, and formatted output instructions. The figure also illustrates the main functionalities available in the compiler interface, such as compilation, lexical analysis, parsing, file loading, and file saving. Figure 24 shows the validation of its execution through all the compiler phases, as well as the output generated in the virtual machine. The remaining figures (25, 26, 27, 28, 29, 30, 31, 32, and 33) show the output after passing through each compiler phase.

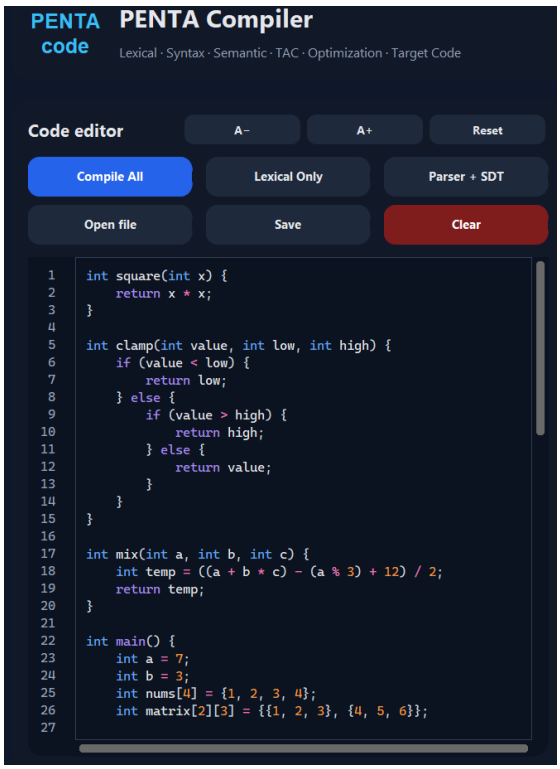

Figure 28: Input for Test Case 3

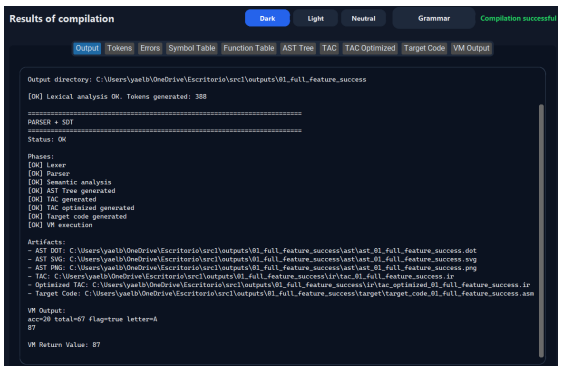

Figure 29: Output for Test Case 3

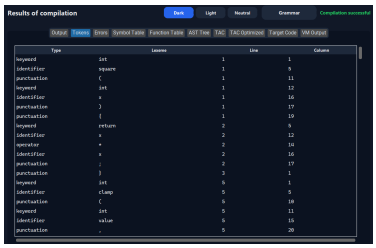

Figure 30: Tokens

Figure 31: Errors

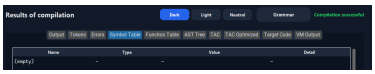

Figure 32: Symbol Table

Figure 33: Function Table

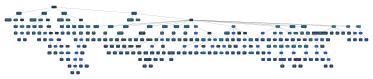

Figure 34: AST Tree

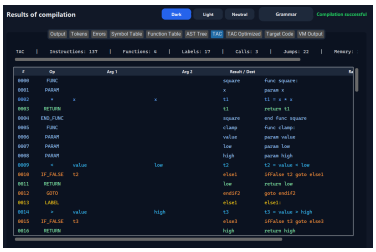

Figure 35: TAC

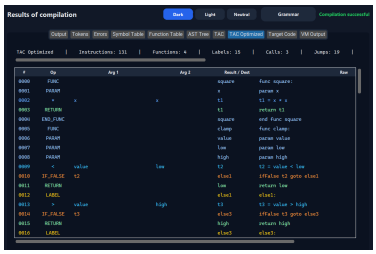

Figure 36: TAC Optimized

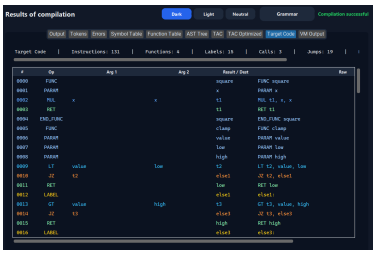

Figure 37: Target Code

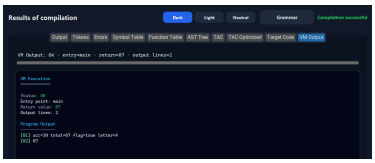

Figure 38: VM Output

4.4 Invalid Test Cases

4.4.1 Test Case 4: Lexical Error (20_lexical_invalid_symbol.c)

Figure 34 shows the input source code containing a lexical error caused by the invalid character @, which does not belong to the language specification supported by the compiler.
Figure 35 presents the program output generated during the lexical analysis phase, where the compiler detects the invalid token and reports a lexical error message.
Finally, Figure 36 highlights the exact location in the source code where the lexical error occurred, allowing the user to identify and correct the invalid character.

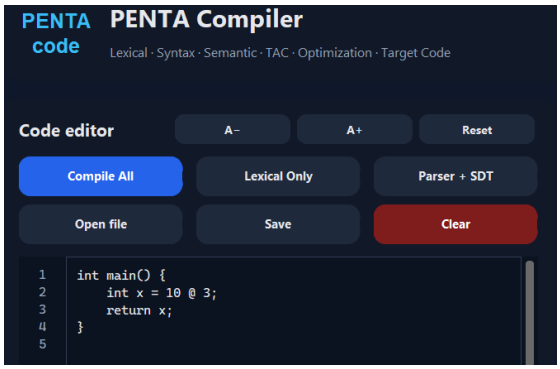

Figure 39: Input for Test Case 4

Figure 40: Output for Test Case 4

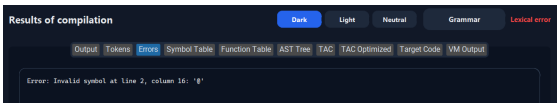

Figure 41: Errors

4.4.2 Test Case 5: Sintax Error

Figure 34 shows the input source code containing a syntax error caused by the missing semicolon after the variable assignment statement.
Figure 35 presents the program output generated during the compilation process. The output indicates that the source code successfully passed the lexical analysis phase; however, the compilation process stopped during the syntax analysis phase due to a syntax error detected by the parser.
Figure 36 displays the tokens and lexemes generated by the lexical analyzer before the syntax analysis process began. This confirms that the lexer correctly identified and classified the elements of the source code.
Finally, Figure 37 highlights the exact location in the source code where the syntax error occurred, allowing the user to identify the missing semicolon and correct the statement.

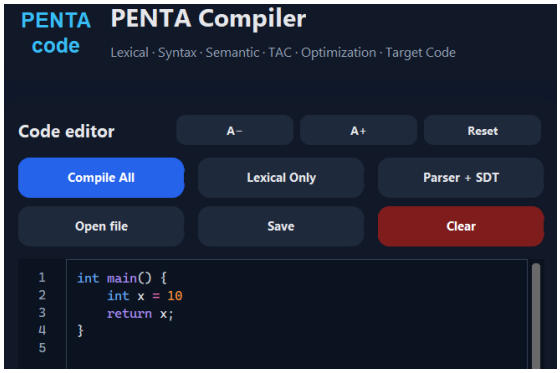

Figure 42: Input for Test Case 5

Figure 43: Output for Test Case 5

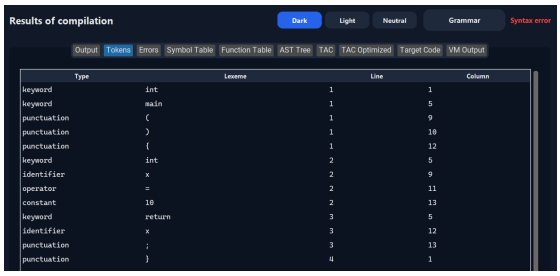

Figure 44: Tokens

Figure 45: Errors

5 Conclusions

The development of the compiler demonstrated how the theoretical foundations of compiler construction can be practically applied to solve the problem of validating and processing C source code. Through the integration of the different compilation phases, it was possible to establish a structured workflow capable of transforming high-level instructions into validated and executable internal representations.

The obtained results highlight the importance of lexical analysis as the initial mechanism for token recognition and classification, since it provides an organized representation of the input that simplifies subsequent compiler stages. Likewise, syntax analysis based on LALR and Shift-Reduce techniques successfully validated the grammatical structure of programs, demonstrating the relevance of context-free grammars and LR automata in the construction of deterministic parsers.

Furthermore, the incorporation of Syntax Directed Translation (SDT) confirmed the necessity of complementing syntactic validation with semantic verification. The implementation of symbol tables, abstract syntax trees, and semantic rules ensured logical consistency within the analyzed programs, detecting errors related to declarations, variable usage, and type consistency before execution.

The valid and invalid test cases demonstrated that the system is capable of accurately identifying and classifying lexical, syntactic, and semantic errors. In addition, the generation of intermediate representations, optimized code, and target code showed the relationship between theoretical compilation phases and their practical implementation within a functional compiler architecture.

Overall, the project demonstrates that the integrated application of concepts such as regular expressions, context-free grammars, LR/LALR parsing, SDT, and semantic structures constitutes an effective solution for modern compiler design, validating the importance of these foundations within the field in which we operate.

References

[1] Alfred V. Aho et al. *Compiladores: Principios, Técnicas y Herramientas*. 2nd ed. Traducción autorizada de Compilers: Principles, Techniques, and Tools, 2nd ed. México: Pearson Educación de México, 2008. ISBN: 9789702611332.

[2] GeeksforGeeks. *Syntax Directed Translation in Compiler Design*. Dec. 2025. URL: https://www.geeksforgeeks.org/compiler-design/syntax-directed-translation-in-compiler-design/.